

高级语言程序设计实验报告

游戏框架设计与实现

南开大学

计算机大类

姓名：刘晋豪

学号：2510670

班级：计算机伯苓班

日期：2026.5.10

目录

1	项目架构与目录结构	3
1.1	高层目录结构	3
1.2	模块依赖	3
2	关键模块与文件清单	4
2.1	角色与能力系统	4
2.1.1	AWarriorBaseCharacter	4
2.1.2	AWarriorHeroCharacter	4
2.2	输入系统	4
2.2.1	UWarriorInputComponent	4
2.2.2	UDataAsset_InputConfig	5
2.3	武器与伤害系统	5
2.3.1	AWarriorWeaponBase	5
2.4	敌人与血量系统	5
2.4.1	AEnemyCharacter	5
2.5	能力系统（数据驱动）	5
2.5.1	UDataAsset_StartUpDataBase	5
2.5.2	UWarriorGameplayAbility	6
2.6	辅助与动画系统	6
3	重要函数与参数说明	6
3.1	PossessedBy (AWarriorBaseCharacter)	6
3.2	SetupPlayerInputComponent (AWarriorHeroCharacter)	6
3.3	BindNativeInputAction (UWarriorInputComponent)	6
3.4	OnWeaponOverlap (AWarriorWeaponBase)	6
3.5	TakeDamage (AEnemyCharacter)	7
3.6	GiveToAbilitySystemComponent (UDataAsset_StartUpDataBase)	7
4	数据与玩法流程（从输入到伤害）	7
5	网络/复制策略（原则）	7
6	构建与依赖	8
7	AI 工具的使用	8

1. 项目架构与目录结构

1.1 高层目录结构

Source/WarriorGame/

Public/ # 头文件与接口（模块按子目录划分）

characters/

WarriorBaseCharacter.h

WarriorHeroCharacter.h

EnemyCharacter.h

EnemyAIController.h

Items/Weapons/

WarriorWeaponBase.h

WarriorHeroWeapon.h

actors/

FireballProjectile.h（可选/示例）

AbilitySystem/

WarriorAbilitySystemComponent.h

Abilities/

WarriorGameplayAbility.h

DataAsset/

StartUpData/

DataAsset_StartUpDataBase.h

Input/

DataAsset_InputConfig.h

Components/Input/

WarriorInputComponent.h

AnimInstances/

若干动画实例头文件

Private/

实现文件（.cpp）

[与 Public 对应子目录结构]

WarriorGame.Build.cs

模块依赖声明

1.2 模块依赖

WarriorGame.Build.cs 包含以下核心模块依赖：- EnhancedInput - GameplayTags
- AIModule - NavigationSystem

2. 关键模块与文件清单

2.1 角色与能力系统

2.1.1 AWarriorBaseCharacter

- 文件位置:
- 头文件: `Public/characters/WarriorBaseCharacter.h`
- 实现: `Private/characters/WarriorBaseCharacter.cpp`
- 功能: 负责创建并持有 `UWarriorAbilitySystemComponent` 与 `UWarriorAttributeSet`, 并在 `PossessedBy` 中初始化 ASC。- 核心方法: `- PossessedBy(AController* NewController):` 调用 `WarriorAbilitySystemComponent->InitAbilityActorInfo(this, this)`, 并(可)通过 `CharacterStartupData` 给 ASC 授能。

2.1.2 AWarriorHeroCharacter

- 文件位置:
- 头文件: `Public/characters/WarriorHeroCharacter.h`
- 实现: `Private/characters/WarriorHeroCharacter.cpp`
- 功能: 玩家角色, 包含摄像机 (SpringArm + Camera)、移动、输入绑定。
- 核心方法:
 - `SetupPlayerInputComponent(UInputComponent* PlayerInputComponent):` 加载 `InputConfigDataAsset->DefaultMappingContext`, 并通过 `UWarriorInputComponent::BindNativeIn` 用 `GameplayTag` 绑定 `Input_Move`、`Input_Look` 等。
 - `Input_Move(const FInputActionValue InputActionValue):` 将二维输入映射到世界方向并调用 `AddMovementInput`。
 - `Input_Look(const FInputActionValue):` 调用 `AddControllerYawInput / AddControllerPitchInput`。

2.2 输入系统

2.2.1 UWarriorInputComponent

- 文件位置:
- 头文件: `Public/Components/Input/WarriorInputComponent.h`
- 实现: `Private/Components/Input/WarriorInputComponent.cpp`
- 核心方法:
 - 模板方法 `BindNativeInputAction(const UDataAsset_InputConfig* InInputConfig, const FGameplayTag InInputTag, ETriggerEvent TriggerEvent, UserObject* ContextObject, CallbackFunc Func):` 从 `DataAsset` 按 `GameplayTag` 找到 `UInputAction` 并 `BindAction` 到回调。

2.2.2 UDataAsset_InputConfig

- 功能：在编辑器中配置 UInputAction 与 FGameplayTag 的对应关系（数据驱动输入）。

2.3 武器与伤害系统

2.3.1 AWarriorWeaponBase

- 文件位置：
- 头文件：Public/Items/Weapons/WarriorWeaponBase.h
- 实现：Private/Items/Weapons/WarriorWeaponBase.cpp - 核心成员：
- WeaponMesh - WeaponCollisionBox - WeaponDamage（默认 15.0） - 核心方法：
- OnWeaponOverlap(UPrimitiveComponent*, AActor* OtherActor, ...):过滤 owner、self 后调用 UGameplayStatics::ApplyDamage(OtherActor, WeaponDamage, nullptr, this, nullptr)。

2.4 敌人与血量系统

2.4.1 AEnemyCharacter

- 文件位置：
- 头文件：Public/characters/EnemyCharacter.h
- 实现：Private/characters/EnemyCharacter.cpp
- 核心属性：
- MaxHealth (EditDefaultsOnly)
- Health (ReplicatedUsing = OnRep_Health)
- WalkSpeed
- PatrolRadius
- 核心方法：
- TakeDamage(float DamageAmount, FDamageEvent const DamageEvent, AController* EventInstigator, AActor* DamageCauser):在服务器上处理减血,调用 OnHealthChanged (BlueprintImplementableEvent) 并在死亡时触发 OnDeath()。 - 复制实现：
- GetLifetimeReplicatedProps 使用 DOREPLIFETIME(AEnemyCharacter, Health)
- OnRep_Health() 在客户端响应血量变化

2.5 能力系统（数据驱动）

2.5.1 UDataAsset_StartUpDataBase

- 数据结构：保存 ActivationGivenAbilities、ReactiveAbilities

- 核心方法:

- GiveToAbilitySystemComponent(UWarriorAbilitySystemComponent* InASCToGive): 遍历并通过 GrantAbilities 给 ASC GiveAbility(FGameplayAbilitySpec)。

2.5.2 UWarriorGameplayAbility

- 继承自 UGameplayAbility

- 覆盖方法: OnGiveAbility 与 EndAbility

- 核心特性: 使用枚举 EWarriorAbilityActivationPolicy 控制是”授予即激活”还是”触发式激活”。

2.6 辅助与动画系统

动画实例(如 UWarriorCharactorAnimInstance 等)读取 GetVelocity()、GetCharacterMovement 的加速度/速度用于动画参数 (BlendSpace)。

3. 重要函数与参数说明

3.1 PossessedBy (AWarriorBaseCharacter)

- 参数: NewController —新的控制器

- 关键实现: 必须在 Possess 之后初始化 ASC, 调用 WarriorAbilitySystemComponent->InitAbilities(this)。

3.2 SetupPlayerInputComponent (AWarriorHeroCharacter)

- 参数: 引擎提供的 PlayerInputComponent

- 行为: 加载 Mapping Context, 将 PlayerInputComponent 转为 UWarriorInputComponent 并按 Tag 进行绑定。

3.3 BindNativeInputAction (UWarriorInputComponent)

- 参数: - InInputConfig: 映射数据 - InInputTag: 查找键 - TriggerEvent: Started/Triggered/Completed - ContextObject - Func: 回调函数指针 - 作用: 把 DataAsset 的 UInputAction 绑定到 C++ 回调。

3.4 OnWeaponOverlap (AWarriorWeaponBase)

- 参数: 详见上文 - 行为: 在服务器上对 OtherActor 调用 ApplyDamage 或对特定类型直接调用死亡逻辑。

3.5 TakeDamage (AEnemyCharacter)

- 参数:
- DamageAmount: 伤害数值
- DamageEvent: 伤害类型
- EventInstigator: 触发者 Controller
- DamageCauser: 伤害源 Actor
- 行为: 仅在服务器上执行减血、触发蓝图事件、在血 ≤ 0 时触发 OnDeath 并 Destroy()。

3.6 GiveToAbilitySystemComponent (UDataAsset_StartUpDataBase)

- 参数:
- InASCToGive: 目标 ASC
- ApplyLevel: 能力等级
- 行为: 通过 GrantAbilities 构建 FGameplayAbilitySpec 并 GiveAbility。

4. 数据与玩法流程（从输入到伤害）

1. 玩家在本地按键 → Enhanced Input 捕获并生成 FInputActionValue。
2. UWarriorInputComponent::BindNativeInputAction 将该 action 调用绑定到 AWarriorHeroCharacter。
3. Input_Move 计算世界方向,调用 AddMovementInput,角色移动由 CharacterMovementComponent 驱动。
4. 当角色进行攻击（由动画触发）时,武器通过 WeaponCollisionBox 的碰撞回调检测命中;在命中时服务器调用 ApplyDamage（或角色/服务器直接处理销毁）。
5. 受击敌人在服务器上 TakeDamage,更新 Health 并通过 replication 同步;客户端 OnRep_Health 收到通知后触发视觉效果（血条/受击特效/死亡动画）。

5. 网络/复制策略（原则）

- 所有权威性数据修改（生成/销毁 Actor、Health 变化、ASC 授能）在服务器执行。
- 使用 SetReplicates(true) 与 SetReplicateMovement(true) 复制运动型 Actor（如投射物、Pawn）。
- 属性复制使用 DOREPLIFETIME 与 ReplicatedUsing 回调以在客户端执行表现逻辑。

- 客户端请求（例如攻击输入或触发能力）通过 Server RPC 实现服务器确认/执行。

6. 构建与依赖

- 构建系统：Unreal Engine 模块依赖系统
- 核心模块依赖：EnhancedInput、GameplayTags、AIModule、NavigationSystem
- 源码位置：Source/WarriorGame/

7. AI 工具的使用

- 豆包：UE 蓝图类创建教学与 DeBug
- ChatGpt5.5mini：VS 内 C++ 代码的修改、DeBug 以及一些玩法的代码实现